

Software Quality Assurance and Management

第5周：测试自动化与持续质量工程

Week 5: Test Automation & Continuous Quality Engineering

By: Fahad Sabah

计算机学院,

北京工业大学, 中国

College of Computer Science,
Beijing University of Technology, China

fahad.sabah@bjut.edu.cn



测试自动化与持续质量工程

Test Automation & Continuous Quality Engineering

What is a test?

A test is like a tiny question we ask the program.

Example: “Is the equipment status Available before checkout?”

If the answer is correct, the test passes. If not, it fails.

What did your command do?

You ran:

```
python -m pytest -q
```

This told Python to run the tests in a module.

What the output means

.. means two tests were run

[100%] means all test questions were finished

2 passed in 5.51s means both tests were happy and everything worked

测试自动化与持续质量工程

Test Automation & Continuous Quality Engineering

Automation testing:

use tools, scripts, and software to run predefined test steps and compare **expected** vs **actual** results.

Goal: reduce repetitive manual testing — **not** eliminate all human testing.

Three common levels:

1. Unit
2. API / integration
3. User interface (UI / GUI)

Why this is useful

- ☐ It proves the web page or the app works the way we expect
- ☐ It helps us catch problems before people use the system
- ☐ Automation can check things quickly and safely

测试自动化与持续质量工程

Manual Testing vs. Automation Testing

Features	Manual Testing	Automation Testing
Accuracy & Reliability	Accuracy is low. Because manual tests are more prone to human error	Accuracy is high as tools and scripts are used
Time required	High as compared to Automation Testing	Relatively Low
Investment Cost	Return of Investment(ROI) is low	Return of Investment(ROI) is high
Usage	Suitable for Exploratory, Usability and Ad hoc Testing	Suitable for Regression Testing, Performance Testing, Load Testing
Human Element	It allows human observation to find out any glitches	No human observation involved
Customer Experience	Helps in improving the customer experience	No guarantee of positive customer experience

测试自动化与持续质量工程

Test Automation & Continuous Quality Engineering

Why automate?

1. **24/7** scheduling (e.g. nightly regression)
2. **Reusable** scripts — no rewrite every cycle
3. **Earlier** defect detection — lower fix cost
4. **Reliable** repetition of standard steps
5. **Higher coverage** in the same time

为什么要自动化？

1. 全天候24小时调度（例如每晚回归）
2. 可重用脚本——每个周期不重写
3. 更早发现缺陷——更低的修复成本
4. 标准步骤的可靠重复
5. 同时期更高的覆盖范围

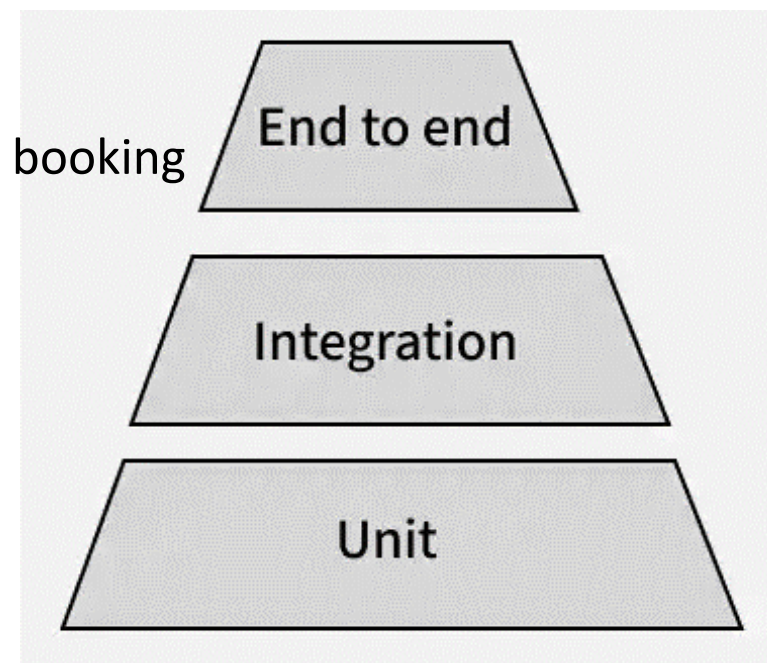
测试金字 Test Pyramid

UI = can a user finish the journey?

Fast, many — overlap rules, 7-day max, no checkout without booking

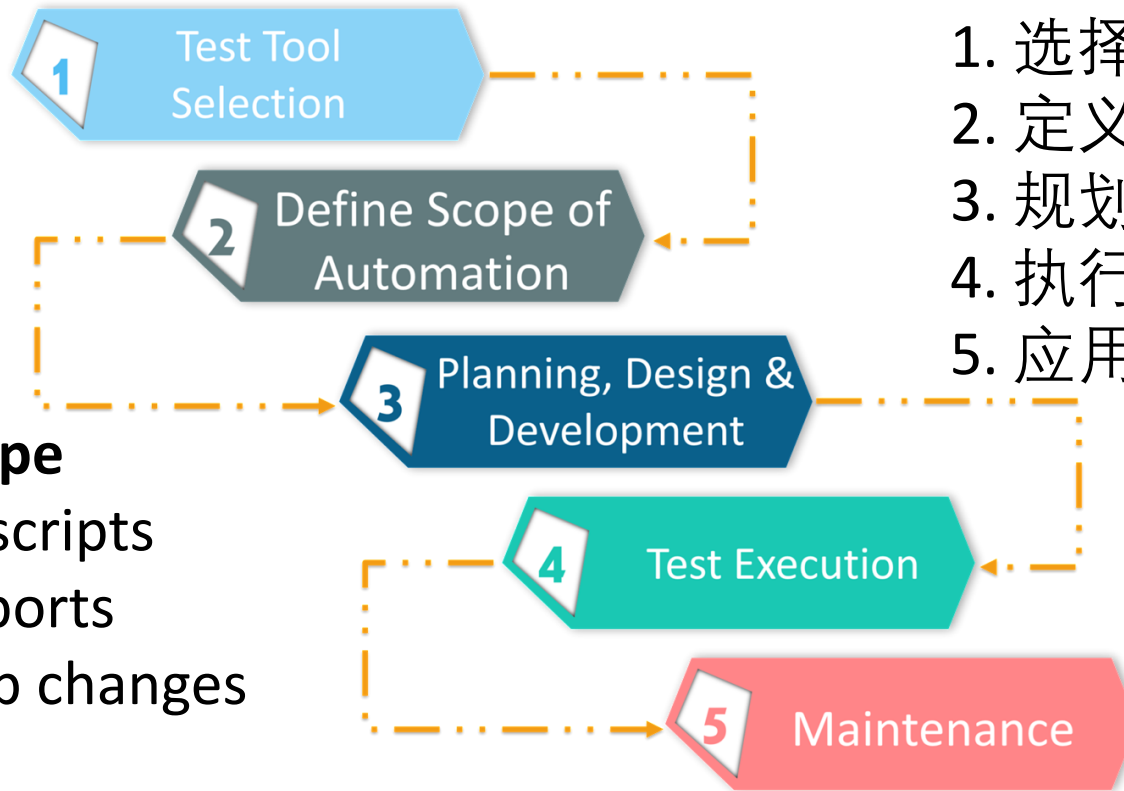
API = wiring; Medium — POST /bookings, checkout flow

Unit = rules; Slow, few —submit booking, date error



从零开始的五步

Five steps from scratch



1. Choose the right **tool**
2. Define automation **scope**
3. **Plan, design, develop** scripts
4. **Execute** and collect reports
5. **Maintain** when the app changes

1. 选择合适的工具
2. 定义自动化范围
3. 规划、设计、开发脚本
4. 执行并收集报告
5. 应用更换时保持维护

自动化什么，不该自动化什么

What to automate vs what not to

Automate:

- Repetitive, tedious, high-regression areas
- Critical business paths (booking success, checkout denied)
- Many data combinations (data-driven)
- Stable cases already run manually at least once

Keep manual:

- Exploratory testing
- Visual / usability
- Rapidly changing requirements
- One-off scenarios

自动化:

- 重复、乏味、高回归的区域
- 关键业务路径（预订成功，结账被拒）
- 多种数据组合（数据驱动）
- 稳定案件至少手动运行过一次

保持手动:

- 探索性测试
- 视觉/可用性
- 快速变化的需求
- 一次性剧本

工具选择检查清单（六题）

Tool selection checklist (six questions)

1. **Ease of Use** – Some of the automation tools are complex and require extensive knowledge, skills, and cost for its usage.
2. **Support for browsers** – The automation tool selected should support cross-browser testing on multiple browsers.
3. **Flexibility** – The automation tool selected should be flexible enough to be adopted easily.
4. **Ease of Analysis** – The automation tool selected should support easy analysis by being able to generate quick dashboards and test reports after the execution.
5. **Cost** – Some of the automation tools are free and open source while some have licensing and other costs.
6. **Support** – The automation tool selected should have adequate community support to resolve the issues(especially for free tools).

测试自动化与持续质量工程

Test Automation & Continuous Quality Engineering

Three approaches

1. **Code-driven:** test functions directly (`pytest` + `lab\_system.py`)
2. **GUI testing:** drive the browser (`Playwright` + `demo\_ui`)
3. **Test automation framework:** POM, separated data, shared libraries (advanced module)

测试自动化与持续质量工程

Test Automation & Continuous Quality Engineering

Requirements → test IDs

ID	Requirement	Layer
BK-02	No overlapping time slots	Unit
CO-01	No checkout without booking	Unit + API
UI-01	End date before start shows error	E2E

测试自动化与持续质量工程

Test Automation & Continuous Quality Engineering

```
Student browser (demo_ui/index.html)    ← UI tests (Playwright)
    |
    | (NOT connected to API – on purpose)
    ▼
FastAPI (src/api.py)                     ← API tests (TestClient)
    |
    | calls
    ▼
Business logic (src/lab_system.py)       ← Unit tests (pytest + fixtures)
    |
    ▼
In-memory dictionaries (no database)
```

“We test rules at the bottom, HTTP wiring in the middle, and what the user sees at the top — same ideas as the test pyramid.”

测试自动化与持续质量工程

Test Automation & Continuous Quality Engineering

- Automation supports quality; it does ****not**** replace exploratory testing or product judgment.
- Next: run your first `pytest`, then calculate pass rate and ROI.

测试自动化与持续质量工程

Test Automation & Continuous Quality Engineering

Unit tests and business logic

How to test business rules in isolation: booking, overlap, max 7 days, checkout, return.

Piece	Role	What to explain
EquipmentStatus	available, maintenance, checked_out	Equipment state machine
BookingStatus	active, cancelled, completed	Reservation lifecycle
MAX_BOOKING_DAYS = 7	Constant	Same rule as UI “7 days” message
LabEquipmentService	Main class	All operations students care about
create_booking	Core	Validates dates, availability, overlap, then stores booking
_has_overlap	Private helper	Intervals overlap if start < other.end and end > other.start
checkout	Needs active booking in time window	Sets equipment to checked_out
return_equipment	After checkout	Sets booking completed, equipment available
cancel_booking	Exists but not covered by tests	Good homework / coverage gap

单元测试一个接一个

Unit tests — one by one

BK-01 — Booking when equipment is available

test_book_when_available_succeeds

Arrange: service + booking_window

Act: create_booking("EQ-001", "student01", start, end)

Assert: Booking exists, status ACTIVE

Happy path; proves the system allows a normal reservation.

test_book_when_in_maintenance_fails

Arrange: Set EQ-001 to MAINTENANCE

Act: Try to book

Assert: ValueError with “not available”

Negative test; equipment state blocks booking.

BK-01 — 设备可用时预订

test_book_when_available_succeeds

安排: 服务+booking_window

幕: create_booking (“EQ-001”, “学生01”, 开始, 结束)

声明: 预订存在, 状态有效

祝你走得愉快;证明系统允许正常预订。

test_book_when_in_maintenance_fails

安排: 将EQ-001设置为维护

行动: 尽量预订

断言: ValueError, 标注为“不可用”

阴性测试;设备、州、区块、预订。

单元测试一个接一个

Unit tests — one by one

BK-02 — No overlapping time slots

test_overlapping_booking_rejected

Arrange: Student01 books EQ-001 for the window

Act: Student02 tries a slot that starts 12 hours later but still overlaps

Assert: ValueError “overlaps”

Same rule as double-booking a microscope; show `_has_overlap` in `lab_system.py`.

test_non_overlapping_same_equipment_ok

Arrange: First booking ends at end

Act: Second booking starts end + 1 hour

Assert: Two different booking IDs

Positive case for BK-02 — back-to-back is allowed.

BK-02 — 无重叠时段

test_overlapping_booking_rejected

安排：学生01为窗户预订EQ-001

行动：学生02尝试了一个12小时后开始但仍然重叠的时段

断言：ValueError“重叠”

和双重预订显微镜一样;`lab_system.py`年里 `_has_overlap`。

test_non_overlapping_same_equipment_ok

安排：首次预订结束

幕：第二次预订开始结束+1小时

声明：两个不同的预订ID。

BK-02阳性病例—允许连续发作。

单元测试一个接一个

Unit tests — one by one

BK-03 / BK-04 — Duration and date order

test_booking_exceeds_max_days

Act: 8-day span

Assert: Error mentions “7 days”

Maps to: MAX_BOOKING_DAYS in code

test_end_before_start_raises

Act: End before start

Assert: “after start”

Maps to: First check in create_booking

BK-03 / BK-04 — 期限与日期顺序

test_booking_exceeds_max_days

幕：8天

断言：错误中提到“7天”

映射到：MAX_BOOKING_DAYS代码

test_end_before_start_raises

幕：开始前结束

断言：“开始后”

地图指向：首次入住时create_booking

单元测试一个接一个

Unit tests — one by one

CO-01 / CO-02 / RT-01 — Checkout and return

test_checkout_without_booking_fails

Act: Checkout at a time with no booking

Assert: “No active booking”

You cannot pick up equipment without a reservation (CO-01).

test_checkout_with_valid_booking

Arrange: Book, then checkout 1 hour after start

Assert: Checkout record exists; equipment CHECKED_OUT

Full CO-02 at unit level.

test_return_sets_available

Arrange: Book → checkout → return

Assert: Equipment AVAILABLE again (RT-01)

test_list_available_excludes_checked_out

Arrange: Checkout EQ-001

Assert: list_available_equipment() has EQ-002 but not EQ-001

“Available list” is a business rule, not just a single field.

CO-01 / CO-02 / RT-01 — 退货及退还

test_checkout_without_booking_fails

法案：一次退房，无需预订

声明：“无活跃预订”

没有预订（CO-01）你不能领取设备。

test_checkout_with_valid_booking

安排：预订，然后在开始后1小时退房

声明：有结账记录;设备CHECKED_OUT

单位级满CO-02。

test_return_sets_available

安排：预订→结账→退货

声明：装备再次可用（RT-01）

test_list_available_excludes_checked_out

安排：查看EQ-001

断言：list_available_equipment () 有EQ-002但

没有EQ-001

“可用列表”是一个商业规则，而不仅仅是单一字段。

单元测试一个接一个

Unit tests — one by one

`test_intentional_fail_demo_for_class`

Skipped — use to show a red bar in pytest

Never leave failing tests in a real pipeline.

Count: 11 tests collected, 10 run + 1 skipped.

`test_intentional_fail_demo_for_class`

跳过 — 用于在pytest中显示红色条

切勿让失败的测试留在真正的流程中。

计数：收集11次测试，10次跑动+1次跳过。

测试自动化与持续质量工程

Test Automation & Continuous Quality Engineering

Live numbers

Metric	Measured (example run)
Total tests	17
Passed	16
Skipped	1 (class demo)
Statement coverage	93.30%
Suite time	~1.4 s

质量指标

Quality metrics

Pass rate [passed ÷ total × 100]

16 passed / 17 collected (1 skipped)

通过率 [通过 ÷ 总分 × 100]

16次通过 / 17场被收集 (1次跳过)

Coverage[Covered lines ÷ total]

~93.3% on src/

覆盖范围[覆盖线路 ÷ 总数]

~93.3%的src/

Suite time

❑ ~1.4 s for unit + API

❑ Compare to 4 h manual regression in ROI exercise

套房时间

~1.4 秒, 单位 + API

与4小时手动回归在ROI锻炼中的比较

Gaps

❑ cancel_booking, some API branches

❑ “Uncovered = homework”

空隙

cancel_booking, 一些API分支

“未揭露=作业”

测试自动化与持续质量工程

API tests — one by one

test_health

GET /health → 200, {"status":"ok"}

Use: “Is the server up?” smoke test.

test_list_equipment

GET /equipment → 200, at least 2 items, IDs like EQ-...

Use: Read path works; seed data present.

test_create_booking_via_api

POST /bookings with ISO datetimes → 201, body has equipment_id, status: active

Use: Happy path through HTTP (BK-01 at API layer).

test_overlapping_booking_returns_400

POST same equipment/time twice, second user → 400, detail contains “overlaps”

Use: Same as test_overlapping_booking_rejected, but asserts HTTP not exception.

测试自动化与持续质量工程

API tests — one by one

test_checkout_without_booking_returns_400

POST /checkout for user with no booking → 400

Use: CO-01 at API layer.

test_checkout_and_return_flow

POST booking → POST checkout with at inside window → POST return → 200, status: available

Use: End-to-end API journey (not UI): book → checkout → return.

UI / E2E

Automating what appears in the browser — selectors, clicks, visible messages.

Element	ID	Purpose
Equipment dropdown	#equipment	EQ-001 / EQ-002
Student ID	#user	Text
Dates	#start, #end	HTML5 date inputs
Submit	button	Triggers JavaScript validation
Message	#message	Success (green) or error (red .error)

JavaScript rules (client-side only):

1. Dates required
2. End must be after start
3. Max 7 days
4. Success: “Booking submitted successfully (demo).”

Lab Equipment Booking

QA Workshop — UI automation demo (client-side validation only)

Equipment

Student ID

Start (date)

End (date)

Lab Equipment Booking

QA Workshop — UI automation demo (client-side validation only)

Equipment

Microscope A



Student ID

student01

Start (date)

06 / 04 / 2026



End (date)

06 / 05 / 2026



Submit booking

Booking submitted successfully (demo).

Lab Equipment Booking

QA Workshop — UI automation demo (client-side validation only)

Equipment

Microscope A



Student ID

student01

Start (date)

06/04/2026



End (date)

06/01/2026



Submit booking

End date must be after start date.

Lab Equipment Booking

QA Workshop — UI automation demo (client-side validation only)

Equipment

Microscope A



Student ID

student01

Start (date)

06 / 04 / 2026



End (date)

06 / 30 / 2026



Submit booking

Booking cannot exceed 7 days.

测试自动化与持续质量工程

Lab Equipment Booking & Checkout Workshop

The tests: what they check

1. ``test_checkout()``

- Start with 5 microscopes
- Checkout one
- Expect:
 - ``available == 4``
 - ``checked_out == 1``

2. ``test_checkout_unavailable()``

- Start with 1 centrifuge
- Checkout it once
- Then try to checkout again
- Expect an error, because none are available

测试自动化与持续质量工程

Lab Equipment Booking & Checkout Workshop

3. `test\_return\_item()`

- Start with 5 microscopes
- Checkout one
- Return it
- Expect:
 - `available == 5`
 - `checked\_out == 0`

4. `test\_return\_empty()`

- Start with 5 microscopes
- Do not checkout anything
- Try to return an item
- Expect an error, because nothing was borrowed

测试自动化与持续质量工程

Lab Equipment Booking & Checkout Workshop

5. ``test_reserve()``

- Have two microscope boxes:
 - one with 3 free
 - one with 2 free
- Try to reserve 4 microscopes
- Expect success
- Expect the returned reservation object to show:
 - ``checked_out == 4``
 - ``available == 0``

6. ``test_reserve_not_enough()``

- Have one microscope box with 1 free
- Try to reserve 2 microscopes
- Expect failure, because there aren't enough total available

测试自动化与持续质量工程

Lab Equipment Booking & Checkout Workshop

Quality

How many checks did we do?

- There are 6 test questions in module 2.

Pass rate

- If all 6 tests pass:
 - ``passed` / total = 6 / 6``
 - ``6 / 6 * 100 = 100%``

What each metric means

- ``passed`` means the brain behaved correctly
- ``total`` means how many questions we asked
- ``pass rate`` tells us how healthy the code is

A friendly comparison

- The code is a toy box and a librarian.
- The tests are children asking:
 - “Can I borrow this?”
 - “Can I return that?”
 - “Can I borrow four from different boxes?”
- If the answers are all correct, the library is working well.

测试工具

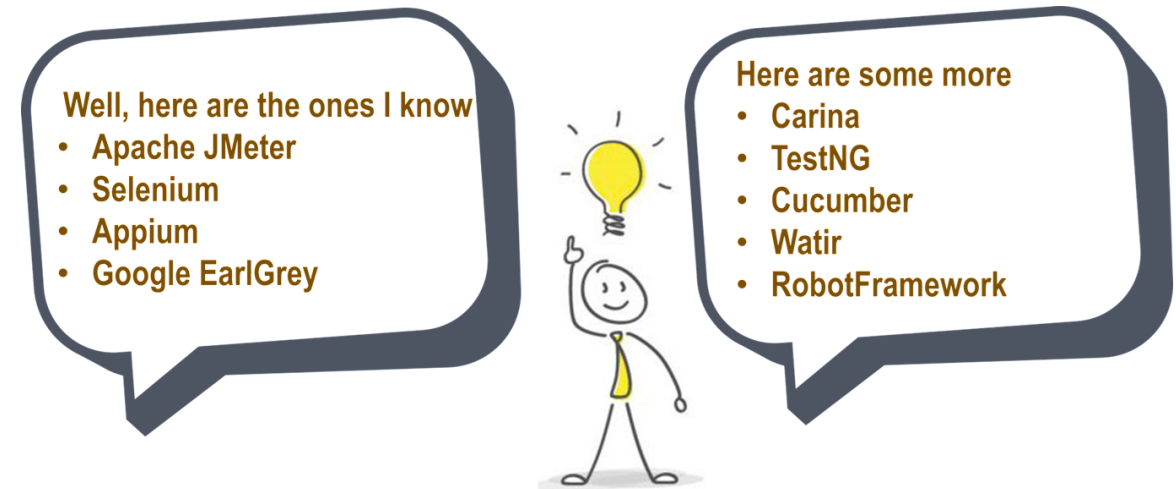
Tools for Testers

Selenium – Selenium is a popular testing framework to perform web application testing across various browsers and platforms like Windows, Mac, and Linux. Get your Selenium Certification and become certified today.

Watir – Watir, pronounced as water, is an open source testing tool made up of Ruby libraries to automate web application testing.

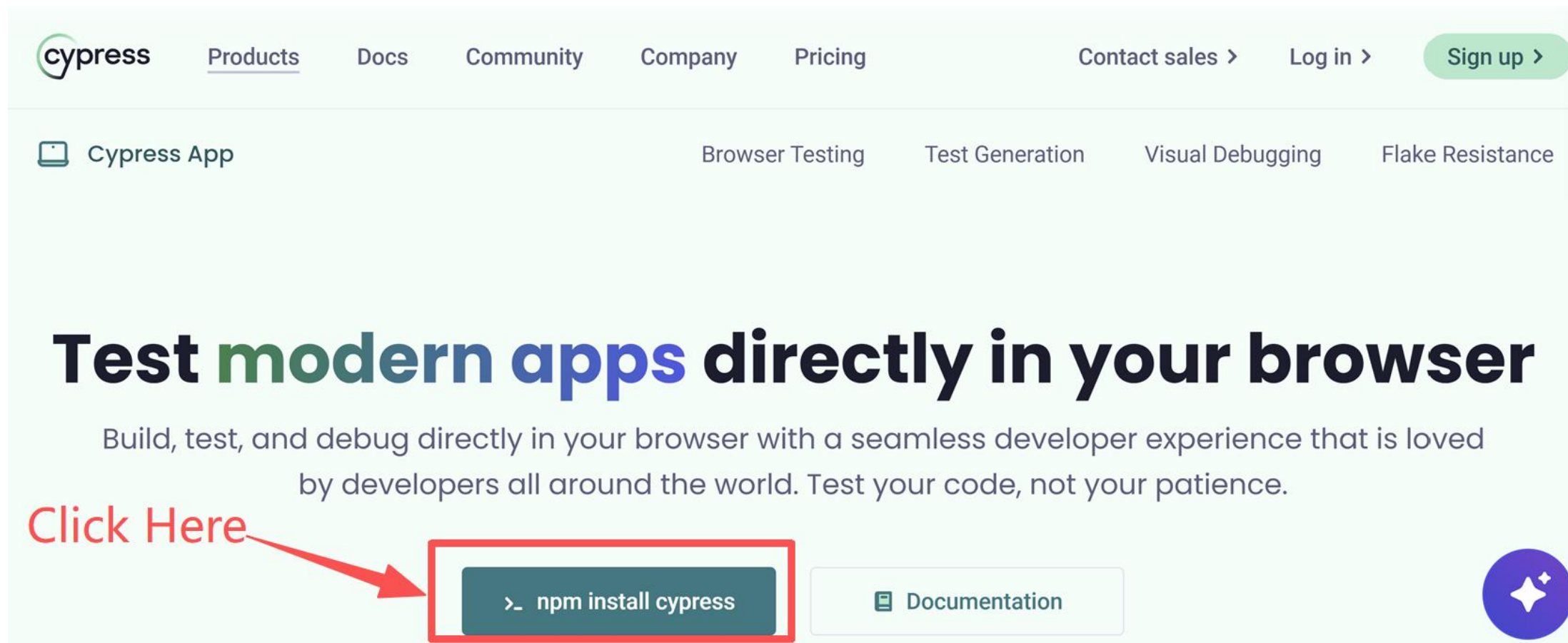
Ranorex – Ranorex is flexible, all in one, GUI testing tool using which you can execute automated tests flawlessly throughout all environments and devices.

Appium – Appium is an Open source mobile test automation software is free and supported by a highly active community of developers and experts.



测试工具: Cypress

Tools for Testers: Cypress



The screenshot shows the Cypress website homepage. The navigation bar at the top includes links for 'Products', 'Docs', 'Community', 'Company', 'Pricing', 'Contact sales >', 'Log in >', and a 'Sign up >' button. Below the navigation bar, there are links for 'Cypress App', 'Browser Testing', 'Test Generation', 'Visual Debugging', and 'Flake Resistance'. The main heading reads 'Test modern apps directly in your browser'. Below this, a subheading states 'Build, test, and debug directly in your browser with a seamless developer experience that is loved by developers all around the world. Test your code, not your patience.' A red arrow points from the text 'Click Here' to a button labeled '> npm install cypress'. To the right of this button is a 'Documentation' button. A purple circular icon with a star is located in the bottom right corner of the main content area.

Test modern apps directly in your browser

Build, test, and debug directly in your browser with a seamless developer experience that is loved by developers all around the world. Test your code, not your patience.

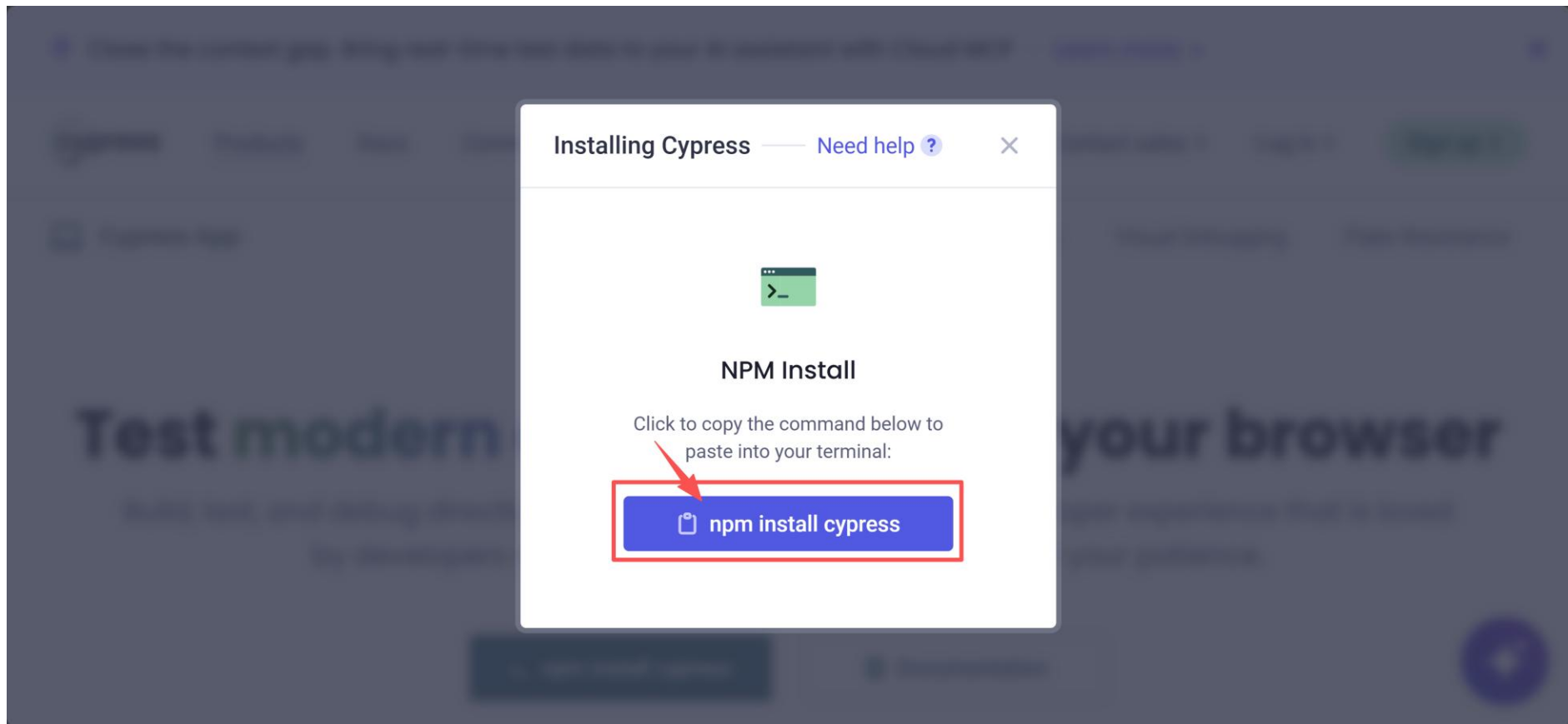
Click Here

> npm install cypress

Documentation

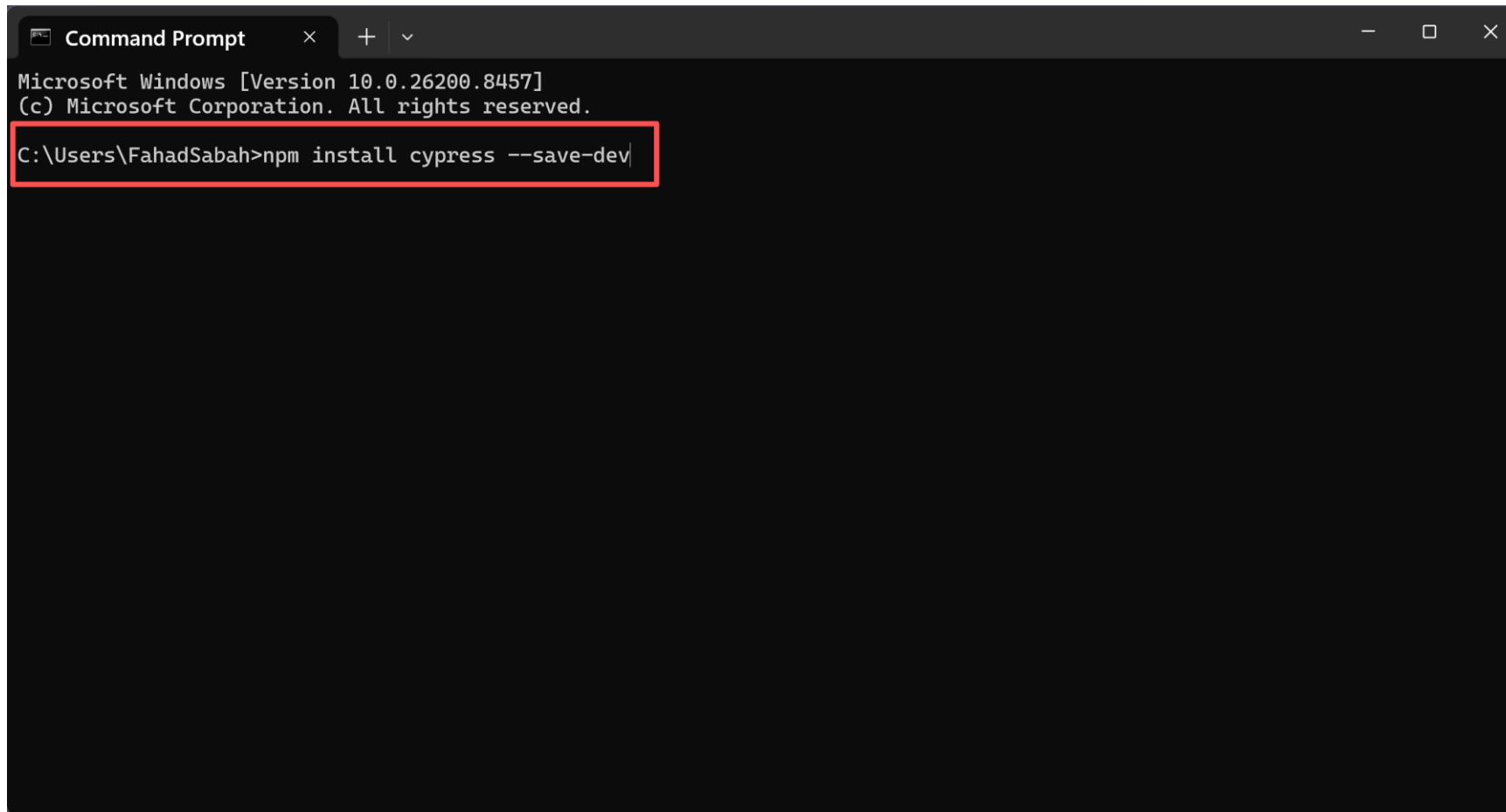
测试工具: Cypress

Tools for Testers: Cypress



测试工具: Cypress

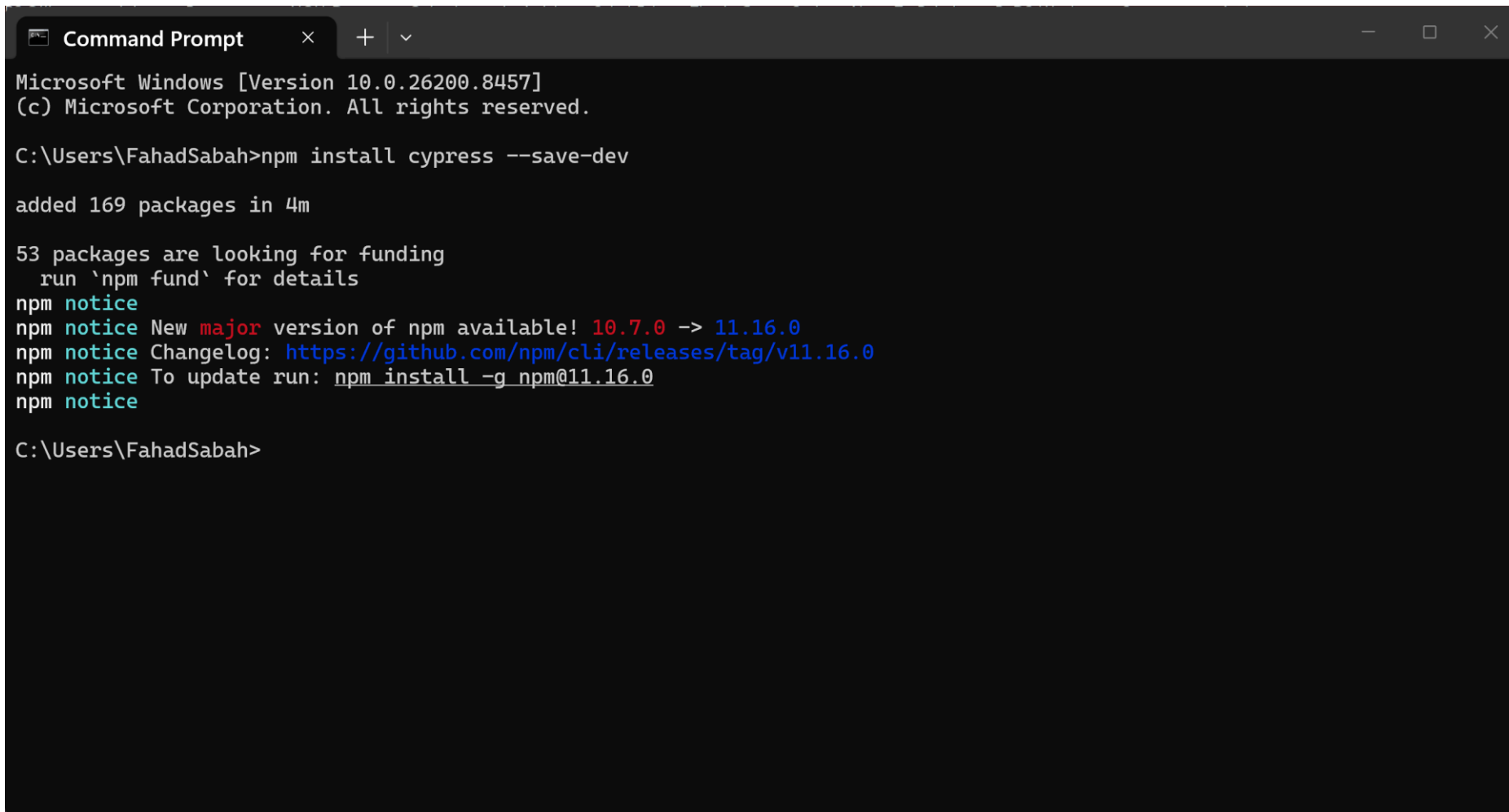
Tools for Testers: Cypress



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.
C:\Users\FahadSabah>npm install cypress --save-dev
```

测试工具: Cypress

Tools for Testers: Cypress



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\FahadSabah>npm install cypress --save-dev

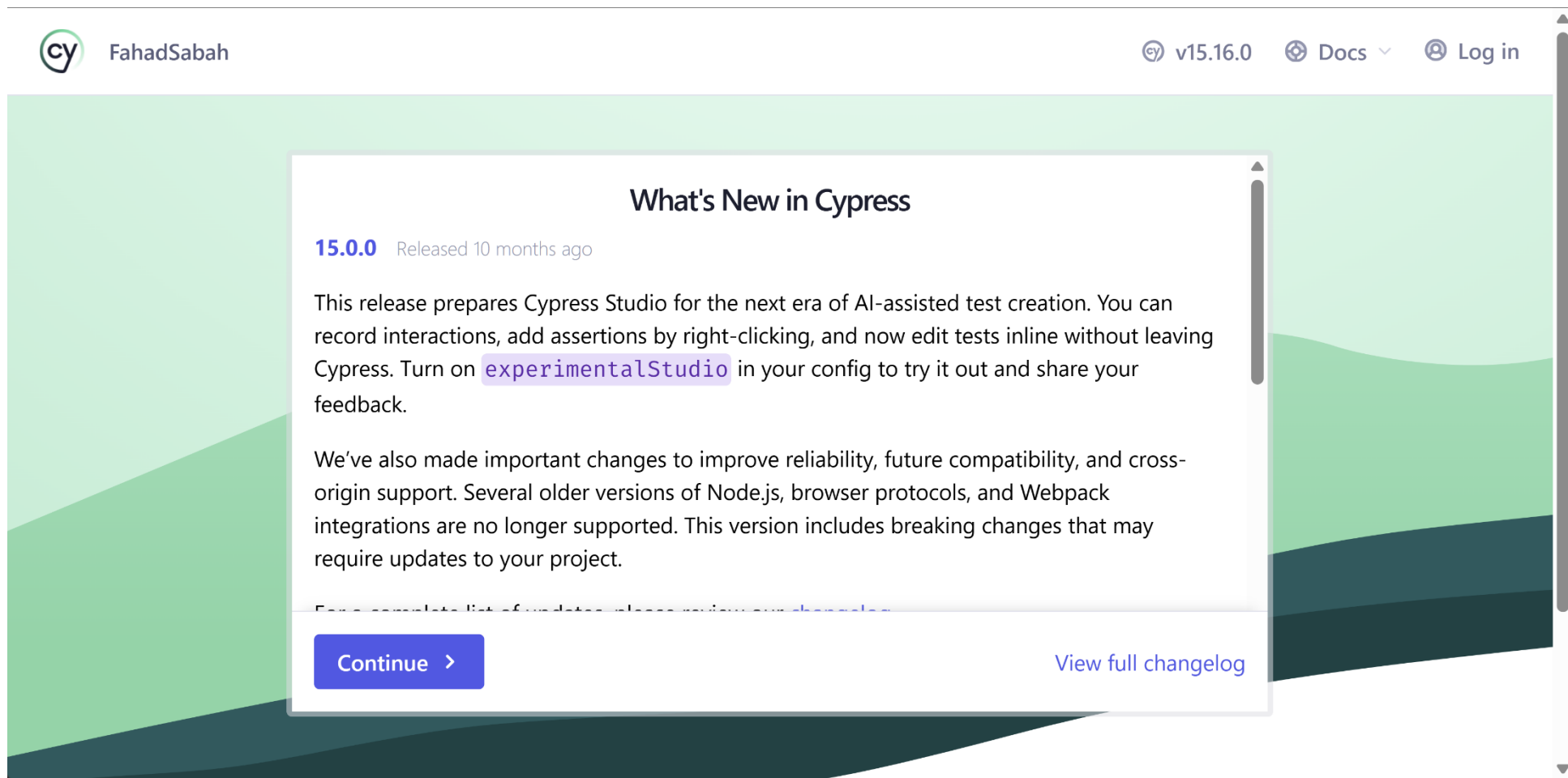
added 169 packages in 4m

53 packages are looking for funding
  run `npm fund` for details
npm notice
npm notice New major version of npm available! 10.7.0 -> 11.16.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.16.0
npm notice To update run: npm install -g npm@11.16.0
npm notice

C:\Users\FahadSabah>
```

测试工具: Cypress

Tools for Testers: Cypress



测试工具: Cypress

Tools for Testers: Cypress



FahadSabah



v15.16.0



Docs



Log in

Welcome to Cypress!

[Review the differences between each testing type →](#)



E2E Testing

Build and test the entire experience of your application from end-to-end to ensure each flow matches your expectations.

Not Configured



Component Testing

Build and test your components from your design system in isolation in order to ensure each state matches your expectations.

Not Configured

Week 5: Test Automation & Continuous Quality Engineering

第5周：测试自动化与持续质量工程

测试工具: Cypress

Tools for Testers: Cypress

 FahadSabah > E2E Testing

 v15.16.0  Docs  Log in

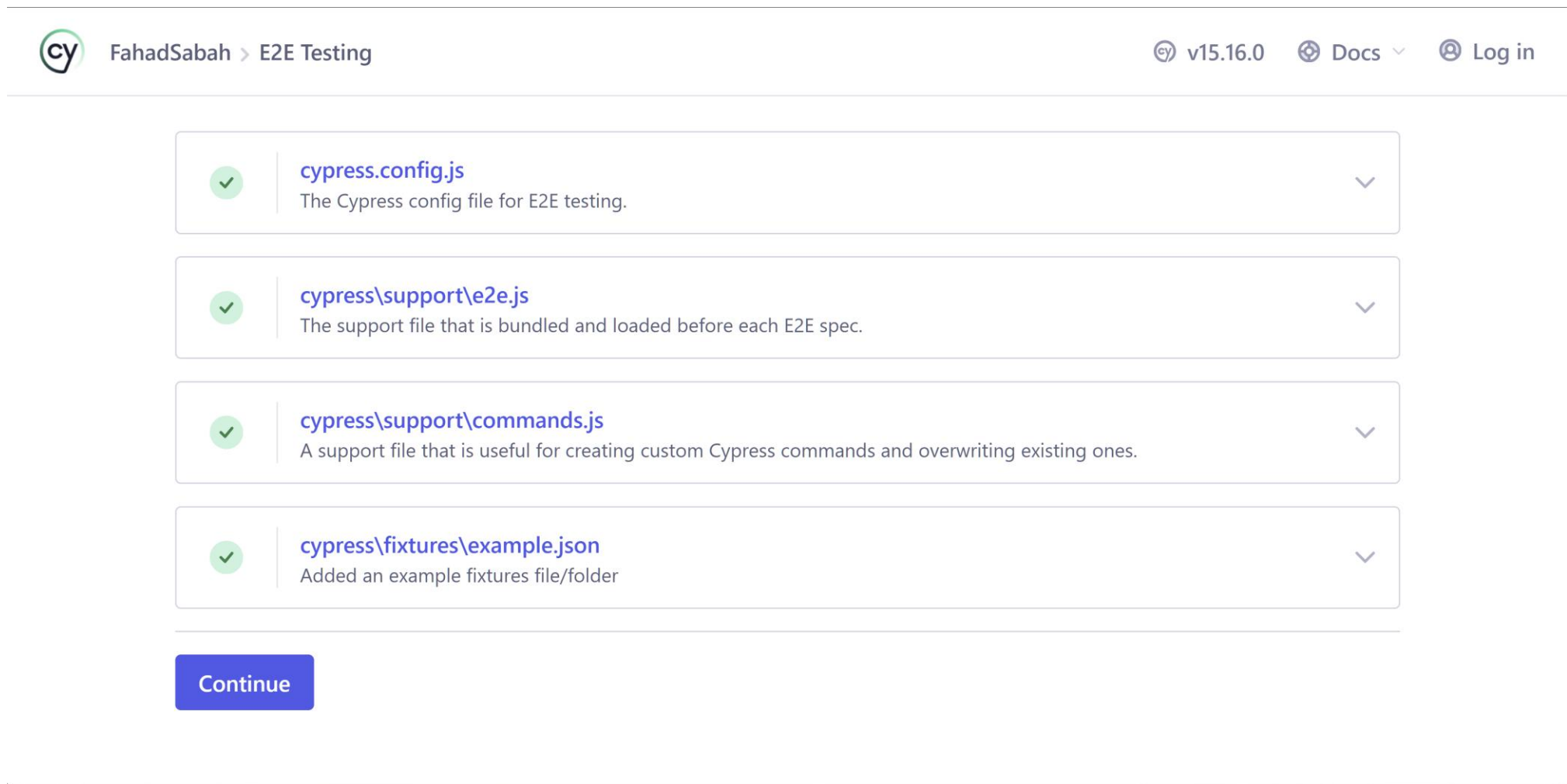
Configuration files

We added the following files to your project:

-  [cypress.config.js](#)
The Cypress config file for E2E testing.
-  [cypress\support\e2e.js](#)
The support file that is bundled and loaded before each E2E spec.
-  [cypress\support\commands.js](#)
A support file that is useful for creating custom Cypress commands and overwriting existing ones.
-  [cypress\fixtures\example.json](#)
Added an example fixtures file/folder

测试工具: Cypress

Tools for Testers: Cypress



测试工具: Cypress

Tools for Testers: Cypress



FahadSabah > E2E Testing

v15.16.0

Docs

Log in

Choose a browser

Choose your preferred browser for E2E testing.



Chrome
v148



Edge
v148



Electron
v138



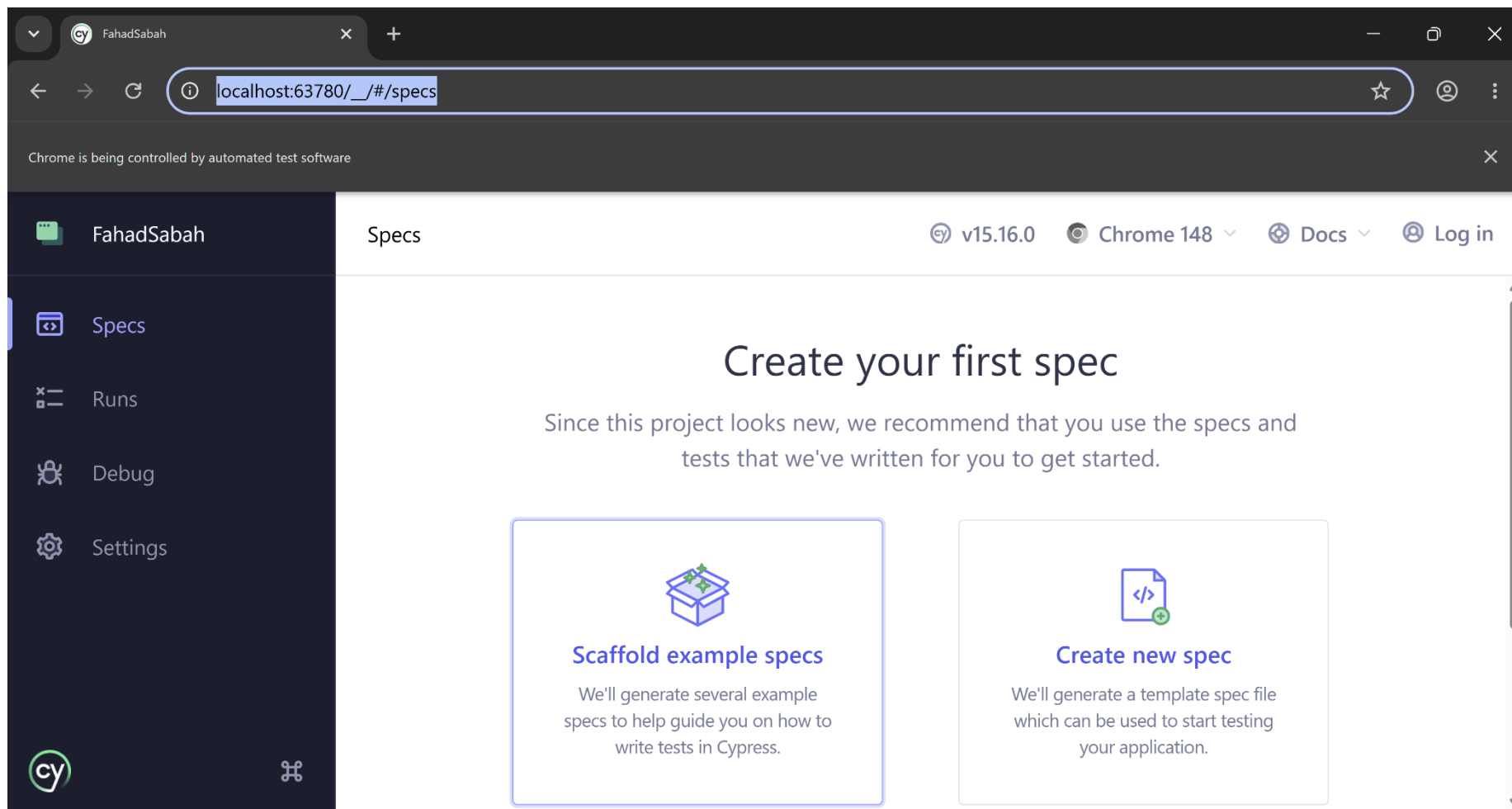
Firefox
v143

 Start E2E Testing in Chrome

← [Switch testing type](#)

测试工具: Cypress

Tools for Testers: Cypress



Week 5: Test Automation & Continuous Quality Engineering

第5周：测试自动化与持续质量工程

LEBCS 演示：端到端实现

LEBCS Demo: End-to-End Implementation

- ☐ Setup and requirements
- ☐ Project file/module explanations
- ☐ End-to-end app flow
- ☐ Data storage and reset button behavior
- ☐ Playwright automation details
- ☐ Workshop activities and discussion questions
- ☐ A dedicated section with software requirement examples and matching tests